B01｜两个程序怎样交换一条工单？

A 册里的函数交接本程序中的值；B01 把交接扩展到另一个运行中的程序：**交去标题与优先级 → 服务创建记录 → 回应交回编号 → 下次用编号读取。** 两端不共享一个 Python 变量。

先认清：谁交给谁，交的是什么？

当前问题	关系或短用法	记到什么程度
哪一边是客户端，哪一边是服务端？	本次发请求的是 client，接收、处理并回应的是 server；同一电脑的两个进程也能承担这两个角色。	认熟 request / response 的方向；角色跟着这次交接判断，不是固定的设备种类。
HTTP、API、JSON 是同一个东西吗？	API 约定怎样使用能力；HTTP 规定请求与回应怎样表达；JSON 表示其中的结构化数据。	**能把三者放回同一次交接。** HTTP 不只传 JSON，API 也不全使用 HTTP。
一条消息怎样读？	请求先看 method 和目标，再看 headers、body；回应先看 status，再看自己的 headers、body。`Content-Type` 声明这份 body 的类型。	逐渐认熟位置，不手拼或背完整报文。请求头和响应头分别属于各自的消息。
`201` 和 `"status":"open"` 都是状态，有什么不同？	`201 Created` 说明这次创建请求的结果；`open` 是工单的业务状态；都不是程序退出码。	**请求结果、资源状态、进程退出分开。** 一次 404 不表示服务进程退出。
为什么不把字典直接交给另一边？	`dict` 是本进程的对象；传送的是 JSON 表示的字节，另一边解析后得到自己的值。	字段相同不等于共用对象；修改客户端字典，不会自动修改服务端记录。

第一次先沿标题、优先级和返回编号看清消息交接，不需要先手写序列化。要查字典怎样变成传送内容时，再认两个方向：json.dumps(值) 产生 JSON 文本，json.loads(文本) 解析出 Python 值；UTF-8 的 `encode` / `decode` 连接文字与字节。这些转换本身不发送请求，实际调用可由 HTTPX 的 `json=` 等工具承担编码。

JSON 的 `true`、`false`、`null` 对应 Python 的 `True`、`False`、`None`，字符串与对象键使用双引号。**合法 JSON 也可能是列表、数字，不一定是当前需要的字典。** 引号和转义交给库处理，不靠拼字符串凑格式。

回看：「HTTP、API 和 JSON 各自负责什么？」「Python 字典看起来像 JSON，为什么还要转换？」。

地址选哪里，动作想做什么?

当前问题	关系或短用法	记到什么程度
完整 URL 里分别在找什么?	`http://127.0.0.1:8000/tickets/T-42?include_history=true`：协议、主机、端口确定服务入口；path 选工单，query 调整本次读取。	能分清目标与选项；编号和端口用实际值，不背样例。
URL path 是磁盘路径吗?	`/tickets/T-42` 由服务解释，不要求磁盘有同名目录。`127.0.0.1` 指当前电脑自身，换电脑就换了“自己”。	**URL 路径不等于文件路径，回环地址不指向另一台电脑。** `https` 使用加密保护的 HTTP 连接。
创建和读取分别怎么表达?	本例 `POST /tickets` 提交 JSON 创建；`GET /tickets/T-42` 读取。`?include_history=true` 要求附带历史，不更换编号。	逐渐用熟 GET 读取、POST 在本例中创建；URL 中的 `true` 是文字，服务处理后才成为 Python 值。
再发一次，效果也只是一次吗?	本例重复 POST 会新建另一条记录。GET 的读取语义不要求改变资源；幂等指重复同一意图的预期效果与一次相同。	**别因标题相同就放心重发创建。** 幂等不要求每次响应内容或状态码都相同。

其他方法按意图查：PUT 通常完整替换，PATCH 局部修改，DELETE 删除；当前配套还实现了 PATCH 状态修改。GET、PUT、DELETE 有幂等语义，POST、PATCH 没有普遍保证。safe 指请求语义不要求改变资源状态，不是保密。方法名不能替代当前 API 的具体约定，也不能让尚未实现的入口自动存在。

回看：「完整地址怎样选中刚才那条工单?」「为什么创建用 POST，读取用 GET?」。

页面显示成功以后，还要看什么证据?

当前问题	关系或短用法	记到什么程度
`200` 就能直接用 body 吗?	成功状态之后，还要检查内容类型、能否解析及所需字段。声明是 JSON，正文也可能损坏或缺字段。	**成功状态不替内容作保证。** B03 再展开每一步。
不熟悉状态码时先看哪里?	2xx 成功；3xx 涉及重定向等处理；4xx 与请求、身份或目标等问题有关；5xx 表示服务未正常完成处理。	认熟大类和正在用的具体码，其余查。404 可能是路径拼错，也可能是工单不存在，数字不能单独定位。
`Accept` 和 `Content-Type` 差在哪?	前者表达希望收到的类型，后者声明当前消息实际发送的类型；希望不等于对方一定满足。	认清两个方向；`Authorization`、Cookie 的完整用法按需查，分享记录前去掉凭据和敏感内容。
怎样核对真正发生的交换?	在 Network 选中本次 POST `/tickets`，对照 URL、method、status、Payload 和 Response；下一次 GET 使用这次返回的 `id`。	逐渐能找对记录并读懂结果；不要把 `/docs` 页面或 `/openapi.json` 的加载当成创建。

地址栏适合发起读取，粘贴 URL 不会自动带 JSON 发 POST。`/docs` 帮助手工构造请求，Network 观察浏览器实际交换，HTTPX 用于 Python 主动调用；它们各解决不同问题。启动服务的完整准备查 B05，不先背配方。

回看：「状态码成功，正文就一定能用了么？」「Header 里除了内容类型，还可能有什么？」「页面写着“创建成功”，怎样看到实际交换？」。

本章看清消息交接；B02 打开接收方，看 **method/path 怎样找到函数，JSON 怎样成为参数，返回值怎样成为响应**。